```java
import java.util.*;

public class Main {

    static class Call {
        int id;
        int arrivalTime;
        int burstTime;
        int remainingTime;
        int completionTime;
        int waitingTime;
        int turnaroundTime;

        Call(int id, int arrivalTime, int burstTime) {
            this.id = id;
            this.arrivalTime = arrivalTime;
            this.burstTime = burstTime;
            this.remainingTime = burstTime;
        }
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of calls: ");
        int n = sc.nextInt();

        if (n <= 0) {
            System.out.println("Number of calls must be positive.");
            sc.close();
            return;
        }

        Call[] c = new Call[n];

        for (int i = 0; i < n; i++) {
            System.out.println("\nEnter details for Call " + (i + 1));
```

```
            System.out.print("Arrival Time: ");
            int at = sc.nextInt();

            System.out.print("Burst Time: ");
            int bt = sc.nextInt();

            if (at < 0 || bt <= 0) {
                System.out.println(
                    "Arrival time must be non-negative and burst time
must be positive."
                );
                sc.close();
                return;
            }

            c[i] = new Call(i + 1, at, bt);
        }

        System.out.print("\nEnter time quantum: ");
        int tq = sc.nextInt();

        if (tq <= 0) {
            System.out.println("Time quantum must be positive.");
            sc.close();
            return;
        }

        Arrays.sort(c, Comparator.comparingInt(p -> p.arrivalTime));

        Queue<Call> queue = new LinkedList<>();

        int currentTime = 0;
        int completed = 0;
        int nextCall = 0;

        ArrayList<String> ganttProcess = new ArrayList<>();
        ArrayList<Integer> ganttTime = new ArrayList<>();
```

```
ganttTime.add(0);

while (completed < n) {

    // Handle CPU idle time
    if (queue.isEmpty() && nextCall < n
            && currentTime < c[nextCall].arrivalTime) {

        currentTime = c[nextCall].arrivalTime;

        ganttTime.set(ganttTime.size() - 1, currentTime);
    }

    // Add arrived calls to the ready queue
    while (nextCall < n &&
            c[nextCall].arrivalTime <= currentTime) {

        queue.add(c[nextCall]);
        nextCall++;
    }

    // Select the next call
    Call current = queue.poll();

    // Execute for one time quantum or remaining burst time
    int executionTime = Math.min(tq, current.remainingTime);

    currentTime += executionTime;
    current.remainingTime -= executionTime;

    ganttProcess.add("C" + current.id);
    ganttTime.add(currentTime);

    // Add newly arrived calls
    while (nextCall < n &&
            c[nextCall].arrivalTime <= currentTime) {
```

```
            queue.add(c[nextCall]);
            nextCall++;
        }

        // Check whether the call has completed
        if (current.remainingTime > 0) {
            queue.add(current);
        } else {
            current.completionTime = currentTime;
            completed++;
        }
    }

    // Calculate turnaround time and waiting time
    double totalWT = 0;
    double totalTAT = 0;

    for (Call p : c) {
        p.turnaroundTime = p.completionTime - p.arrivalTime;
        p.waitingTime = p.turnaroundTime - p.burstTime;

        totalWT += p.waitingTime;
        totalTAT += p.turnaroundTime;
    }

    // Display Gantt chart
    System.out.println("\n========== GANTT CHART ==========");

    System.out.print("|");

    for (String process : ganttProcess) {
        System.out.printf("  %s  |", process);
    }

    System.out.println();

    for (int time : ganttTime) {
        System.out.printf("%-8d", time);
```

```
        }

        // Display scheduling table
        System.out.println("\n\n========== SCHEDULING TABLE
==========");

        System.out.println("Call\tAT\tBT\tCT\tTAT\tWT");

        // Display calls in their original ID order
        Arrays.sort(c, Comparator.comparingInt(p -> p.id));

        for (Call p : c) {
            System.out.println(
                "C" + p.id + "\t" +
                p.arrivalTime + "\t" +
                p.burstTime + "\t" +
                p.completionTime + "\t" +
                p.turnaroundTime + "\t" +
                p.waitingTime
            );
        }

        // Display averages
        System.out.printf(
            "\nAverage Turnaround Time: %.2f\n",
            totalTAT / n
        );

        System.out.printf(
            "Average Waiting Time: %.2f\n",
            totalWT / n
        );

        sc.close();
    }
}
```